

## Phase 3: Schema Design

- **Member**(*MemberID, FirstName, LastName, Email, Phone, JoinDate*)  
Primary Key: MemberID
- **Book**(*BookID, Title, Author, Category, TotalCopies, AvailableCopies*)  
Primary Key: BookID
- **Staff**(*StaffID, FirstName, LastName, Email, JobTitle, HireDate*)  
Primary Key: StaffID
- **Loan**(*LoanID, MemberID, BookID, StaffID, BorrowDate, DueDate, ReturnDate*)  
Primary Key: LoanID  
Foreign Key: MemberID references Member  
Foreign Key: BookID references Book  
Foreign Key: StaffID references Staff
- **Fine**(*FineID, LoanID, Amount, Paid, PaidDate*)  
Primary Key: FineID  
Foreign Key: LoanID references Loan

### Reasoning

I applied the composite attribute rule by splitting `full_name` into `first_name` and `last_name` for `Member` and `Staff`, since a composite attribute cannot be queried on its individual parts if stored as one column. There were no multivalued attributes in this design. For the three one-to-many relationships feeding into `Loan`, I placed the foreign keys directly in `Loan` rather than creating separate relationship schemas, since `Loan` already exists as its own entity and totally participates in all three relationships, meaning there are no null values to avoid by splitting them out further.

For the one-to-one relationship between `Loan` and `Fine`, I placed the foreign key in `Fine` rather than `Loan`, to place it in the entity with total participation. `Fine` cannot exist without a loan, while most loans never produce a fine, so putting `LoanID` in `Fine` avoids a column full of nulls in `Loan`. I also did not need a surrogate key discussion here, since `LoanID` is already a simple single-column primary key rather than a large composite key.